

Algoritmos y Estructura de Datos

Guía integral para el final — UTN FRBA (1er año)

Preparación para el final del 24/07

1. Cómo es el final y cómo se corrige

Reglas generales que la cátedra espera siempre

2. Punteros y memoria dinámica en C++

2.1 Conceptos clave

2.2 Paso de punteros/arrays por parámetro

2.3 Ejercicio de trace resuelto (el que mandaste, final 26/02/2024)

2.4 Errores clásicos que la cátedra pone en V o F

3. Arrays / Vectores y Matrices

3.1 Vector de registros (struct)

3.2 Actualizar un vector a partir de una lista (patrón de examen)

3.3 Matrices (arrays 2D)

4. Listas enlazadas

4.1 Nodo y TDA básico

4.2 Operaciones fundamentales

4.3 Vector de listas (patrón CONAIISI)

4.4 Listas dobles y circulares (por si aparecen)

5. Pilas (Stack) y Colas (Queue)

5.1 Pila (LIFO — el último que entra es el primero que sale)

5.2 Cola (FIFO — el primero que entra es el primero que sale)

5.3 Ejercicio resuelto: intersección de una pila y una lista (final 26/02/2024)

6. Archivos

6.1 Conceptos

6.2 Patrón de apertura y recorrido (pseudocódigo estilo cátedra)

6.3 Ejercicio resuelto: función de Derivación (final 22/07/2024, CONAIISI)

6.4 Ejercicio resuelto: función Dictamen (final 29/07/2024, CONAIISI)

7. Recursividad (por si toma un ejercicio corto)

8. Checklist de repaso para el 24/07

9. Recursos que armaste / que tenés disponibles

1. Cómo es el final y cómo se corrige

El final de Algoritmos y Estructura de Datos (cátedra UTN FRBA) es **escrito**, individual, sin materiales, y evalúa la capacidad de **diseñar funciones en pseudocódigo/C++** que resuelvan un problema, más una parte de **trazado de código** (decir qué imprime un programa) y preguntas de **V o F** justificadas.

Estructura típica (según los finales que mandaste, 2024):

- **Problema grande “de cátedra”** (tipo CONAIIISI, cuentas bancarias, etc.), dividido en 2-3 funciones a programar. Vale la mayor parte de la nota.
- **Ejercicio de trace de código C++** con punteros y arrays dinámicos: hay que decir qué imprime, o justificar por qué no compila.
- **V o F** con justificación, y en algunos finales **resta puntos si te equivocás** (ojo con adivinar).

Reglas generales que la cátedra espera siempre

1. **Declarar todas las estructuras de datos** que uses (struct, TDAs), aunque no te las den armadas.
2. **Usar los parámetros adecuados** (por valor vs. por referencia) y ser explícito con los tipos.
3. Si te dan funciones “de la cátedra” para usar sin desarrollar (`insertarOrdenado` , `push` , `pop` , `queue` , `unQueue` , `buscarEnLista`), **respetá el prototipo tal cual lo dieron** — no las reinventes, y dejalas declaradas en el pseudocódigo aunque no las codifiques.
4. Mantené la **precondición de orden** de archivos/listas si el enunciado lo pide (ej: “el archivo debe quedar ordenado por categoría y fecha”).
5. Separá bien las **etapas del problema** — normalmente el enunciado ya viene fraccionado en pasos, y te piden una función por paso.

2. Punteros y memoria dinámica en C++

Es la base de todo lo demás (listas, pilas, colas, árboles se arman con punteros).

2.1 Conceptos clave

- Un **puntero** guarda una dirección de memoria. `int *p;` declara un puntero a entero (todavía sin inicializar → apunta a “cualquier lado”).
- `p = new int;` reserva memoria dinámica (en el heap) para un entero y `p` apunta ahí.
- `p = new int[5];` reserva un **array dinámico** de 5 enteros contiguos.
- `*p` es el **valor apuntado** (desreferenciar). `p[i]` equivale a `*(p+i)`.
- `delete p;` libera memoria de un solo elemento. `delete [] p;` libera un array. **Siempre liberar lo que reservás con `new`**, si no, memory leak.
- `NULL` (o `nullptr`) indica que el puntero no apunta a nada válido todavía.

2.2 Paso de punteros/arrays por parámetro

Cuando pasás un array a una función, en realidad estás pasando **la dirección del primer elemento** — por eso los cambios que hace la función sobre el array **sí impactan afuera**, aunque no uses `&`.

```
void proc(int v[], int len) {  
    v[0] = 10; // modifica el array original  
}
```

Esto es clave para entender los ejercicios de “qué imprime el programa”: un array (o un puntero) pasado como parámetro **no se copia**, se comparte.

2.3 Ejercicio de trace resuelto (el que mandaste, final 26/02/2024)

```
void proc (int v[], int len){  
    *v = 10;  
}  
  
int main() {  
    int *p = NULL;  
    int x = 30;  
    p = new int[3];  
    p[1] = x;           // p[1] = 30  
    proc(p, 3);         // *v = *p = 10 -> p[0] = 10  
    p[2] = *(p+1) + x;   // p[2] = p[1] + x = 30 + 30 = 60  
    for (int i=0; i<3; i++)  
        cout << *(p+i) << " " << endl;  
    delete []p;  
    return 0;  
}
```

Salida: 10 30 60 (cada uno en su línea, por el `endl`).

Razonamiento paso a paso (así hay que justificarlo en el final):

1. `p` apunta a un array dinámico de 3 enteros, sin inicializar.
2. `p[1] = x;` → `p[1] = 30`.
3. `proc(p,3)` recibe `p` como `v` (comparten memoria). `*v = 10` equivale a `v[0] =`

10, o sea `p[0] = 10`.

4. `p[2] = *(p+1) + x;` → `p[1]` sigue siendo 30, entonces `p[2] = 30 + 30 = 60`.

5. Se imprime `p[0], p[1], p[2]` → 10 30 60.

Segundo bloque del mismo ejercicio:

```
int main() {
    int *p;
    int x = 10;
    p = new int [10];
    p[0] = 20;
    p[1] = 21;
    p[2] = *p;          // p[2] = p[0] = 20
    p[3] = *(p+1);      // p[3] = p[1] = 21
    p[4] = *p + *(p+1); // p[4] = p[0] + p[1] = 20 + 21 = 41
    cout << *(p+4) << endl; // imprime p[4]
    return 0;
}
```

Salida: 41

2.4 Errores clásicos que la cátedra pone en V o F

- “La manera correcta de enviar un vector por referencia es `int *vec[]`” → **FALSO**. Eso declara un *array de punteros a int*, no un puntero a un array. Por parámetro, un vector se pasa como `int vec[]` o `int *vec` (ya viaja “por referencia” de hecho, porque es una dirección).
- Usar un puntero sin inicializar (`int *p;` y desreferenciarlo sin `new` ni asignación) es **comportamiento indefinido** → si aparece en un trace, hay que decir que el programa **no tiene salida determinística / puede fallar**.
- Olvidarse el `delete []` de un array reservado con `new []` → memory leak (no rompe el programa pero es mala práctica, y la cátedra lo suele marcar).

3. Arrays / Vectores y Matrices

3.1 Vector de registros (struct)

Patrón muy típico (aparece en el ejercicio de “Salos de cuentas bancarias” que mandaste):

```
struct Cuenta {
    int numeroCuenta;
    int cantidadTransacciones;
    float saldo;
};

// Vector estático de tamaño fijo conocido (máx. 150 cuentas)
Cuenta saldos[150];
int cantidadCuentas; // cantidad de posiciones realmente usadas
```

Búsqueda dentro de un vector no ordenado: recorrido lineal comparando clave (ej: número de cuenta) hasta encontrarlo o llegar al final.

```
int buscarCuenta(Cuenta saldos[], int cantidadCuentas, int numeroBuscado) {
    int i = 0;
    bool encontrado = false;
    while (i < cantidadCuentas && !encontrado) {
        if (saldos[i].numeroCuenta == numeroBuscado)
            encontrado = true;
        else
            i++;
    }
    if (encontrado) return i;
    return -1; // no encontrado
}
```

3.2 Actualizar un vector a partir de una lista (patrón de examen)

Ejercicio tipo: “tenés un vector de saldos y una lista de transacciones; actualizá el vector recorriendo la lista”.

```
void actualizarSaldos(Cuenta saldos[], int cantidadCuentas, NodoTransaccion
*listaTransacciones) {
    NodoTransaccion *actual = listaTransacciones;
    while (actual != NULL) {
        int pos = buscarCuenta(saldos, cantidadCuentas, actual->numeroCuenta);
        if (pos != -1) {
            saldos[pos].saldo += actual->monto;
            saldos[pos].cantidadTransacciones++;
        }
        actual = actual->siguiente;
    }
}
```

Idea general para “recorrer estructura A y actualizar estructura B”: siempre pensar (1) cómo recorro A de punta a punta, (2) cómo busco/ubico el registro correspondiente en B, (3) qué campos actualizo y con qué operación (sumar, reemplazar, contar).

3.3 Matrices (arrays 2D)

```
const int FILAS = 5, COLUMNAS = 5;
int matriz[FILAS][COLUMNAS];

// Recorrido típico
for (int i = 0; i < FILAS; i++)
    for (int j = 0; j < COLUMNAS; j++)
        matriz[i][j] = 0;
```

Pasaje por parámetro de matriz de tamaño fijo: `void f(int m[FILAS][COLUMNAS])` — la primera dimensión puede omitirse, la segunda **no**.

4. Listas enlazadas

4.1 Nodo y TDA básico

```
struct Nodo {
    TipoDato dato;
    Nodo *siguiente;
};
typedef Nodo* Lista; // puntero al primer nodo (o NULL si está vacía)
```

4.2 Operaciones fundamentales

Insertar al principio:

```
void insertarAlPrincipio(Lista &l, TipoDato valor) {
    Nodo *nuevo = new Nodo;
    nuevo->dato = valor;
    nuevo->siguiente = l;
    l = nuevo;
}
```

Insertar ordenado (el prototipo que casi siempre te dan “hecho” en la cátedra, pero conviene saber programarlo):

```
void insertarOrdenado(Lista &l, TipoDato valor) {
    Nodo *nuevo = new Nodo;
    nuevo->dato = valor;
    if (l == NULL || valor <= l->dato) {
        nuevo->siguiente = l;
        l = nuevo;
    } else {
        Nodo *actual = l;
        while (actual->siguiente != NULL && actual->siguiente->dato < valor)
            actual = actual->siguiente;
        nuevo->siguiente = actual->siguiente;
        actual->siguiente = nuevo;
    }
}
```

Recorrer / buscar:

```
Nodo* buscarEnLista(Lista l, TipoClave clave) {
    Nodo *actual = l;
    while (actual != NULL && actual->dato.clave != clave)
        actual = actual->siguiente;
    return actual; // NULL si no está
}
```

Eliminar un nodo (con puntero auxiliar “anterior”):

```
void eliminar(Lista &l, TipoClave clave) {
    Nodo *actual = l, *anterior = NULL;
    while (actual != NULL && actual->dato.clave != clave) {
        anterior = actual;
        actual = actual->siguiente;
    }
    if (actual != NULL) {
```



```
    if (anterior == NULL) l = actual->siguiente; // era el primero
    else anterior->siguiente = actual->siguiente;
    delete actual;
}
}
```

4.3 Vector de listas (patrón CONALLSI)

Muy usado cuando hay **categorías**: un vector donde cada posición es la cabecera de una lista distinta.

```
const int CANTIDAD_CATEGORIAS = 5;
Lista trabajosPorCategoria[CANTIDAD_CATEGORIAS]; // vector de 5 listas
```

Para trabajar con esto: la categoría te da el índice del vector (`categoria - 1` si van de 1 a 5), y dentro buscás/insertás en **esa** lista puntual con las funciones de siempre.

4.4 Listas dobles y circulares (por si aparecen)

- **Doblemente enlazada**: cada nodo tiene `anterior` y `siguiente` → permite recorrer en ambos sentidos, borrar sin necesitar puntero auxiliar.
- **Circular**: el último nodo apunta al primero en vez de a `NULL` → útil para turnos rotativos (ej. simulación de round-robin).

5. Pilas (Stack) y Colas (Queue)

5.1 Pila (LIFO — el último que entra es el primero que sale)

TDA típico de cátedra:

```
struct NodoPila {
    TipoDato dato;
    NodoPila *siguiente;
};
typedef NodoPila* Pila;

void push(Pila &p, TipoDato valor) {
    NodoPila *nuevo = new NodoPila;
    nuevo->dato = valor;
    nuevo->siguiente = p;
    p = nuevo;
}

TipoDato pop(Pila &p) {
    TipoDato valor = p->dato;
    NodoPila *aux = p;
    p = p->siguiente;
    delete aux;
    return valor;
}

bool pilaVacía(Pila p) { return p == NULL; }
```

5.2 Cola (FIFO — el primero que entra es el primero que sale)

```
struct NodoCola {
    TipoDato dato;
    NodoCola *siguiente;
};
struct Cola {
    NodoCola *frente;
    NodoCola *fondo;
};

void queue(Cola &c, TipoDato valor) { // encolar (push atrás)
    NodoCola *nuevo = new NodoCola;
    nuevo->dato = valor;
    nuevo->siguiente = NULL;
    if (c.frente == NULL) c.frente = nuevo;
    else c.fondo->siguiente = nuevo;
    c.fondo = nuevo;
}

TipoDato unQueue(Cola &c) { // desencolar (pop adelante)
    TipoDato valor = c.frente->dato;
    NodoCola *aux = c.frente;
    c.frente = c.frente->siguiente;
    if (c.frente == NULL) c.fondo = NULL;
    delete aux;
    return valor;
}
```

5.3 Ejercicio resuelto: intersección de una pila y una lista (final 26/02/2024)

“Desarrolle una función que retorne una lista ordenada a partir de la intersección de 1 pila y 1 lista.”

Precondiciones propuestas: la pila y la lista contienen datos del mismo tipo comparable (ej. enteros); la lista de entrada puede estar desordenada; se define “intersección” como los elementos presentes en ambas estructuras.

Idea de resolución: 1. Recorrer la pila haciendo `pop` (esto la vacía — aclarar la precondición o trabajar sobre una copia si el enunciado exige no destruirla). 2. Por cada elemento desapilado, buscarlo en la lista con `buscarEnLista`. 3. Si está presente, insertarlo **ordenado** en la lista resultado con `insertarOrdenado` (evitando duplicados si corresponde).

```
Lista interseccion(Pila &p, Lista l) {
    Lista resultado = NULL;
    while (!pilaVacia(p)) {
        TipoDato valor = pop(p);
        if (buscarEnLista(l, valor) != NULL) {
            if (buscarEnLista(resultado, valor) == NULL) // evita duplicados
                insertarOrdenado(resultado, valor);
        }
    }
    return resultado;
}
```

Este es el tipo de respuesta que la cátedra espera: **estructuras declaradas, precondiciones explícitas, uso de las funciones “de librería” permitidas, y complejidad justificada si te la piden.**

6. Archivos

Es uno de los tres temas que la cátedra marca explícitamente (“archivos, array, estructuras enlazadas”), así que suele aparecer en el problema grande.

6.1 Conceptos

- **Archivo secuencial:** se recorre de principio a fin, no se puede “saltar” a un registro puntual sin leer los anteriores. Para modificarlo manteniendo el orden, generalmente se **reescribe completo** (archivo temporal + reemplazo).
- **Archivo de acceso directo/aleatorio (registros de tamaño fijo):** permite ir directo a la posición `n` calculando el offset (`n * sizeof(registro)`), típico con `fseek`.
- **Mantener ordenado un archivo:** al insertar, hay que ubicar la posición correcta (por clave, ej. categoría + fecha) y reacomodar, o bien insertar en un temporal en el orden correcto y reemplazar el original.

6.2 Patrón de apertura y recorrido (pseudocódigo estilo cátedra)

```
struct RegistroTrabajo {
    char titulo[50];
    int categoria;          // 1..5
    char correoResponsable[50];
    int idEvaluadores[3];
    int resultados[3];      // notas de cada evaluador
    char dictamen[40];
};

void presentarTrabajo(FILE *archivoTrabajos, RegistroTrabajo nuevoTrabajo) {
    // 1. Abrir archivo para lectura/escritura (ya dado)
    // 2. Recorrer buscando la posición según categoría y fecha (orden pedido)
    // 3. Insertar manteniendo el orden (archivo temporal si es secuencial)
    // 4. Cerrar / dejar consistente al final de la jornada
}
```

Lo que la cátedra evalúa acá no es la sintaxis exacta de `fopen / fread` (eso varía), sino que:

- Definas bien el **registro** (struct) con todos los campos del enunciado.
- Repetas la **precondición de orden** (clave compuesta: categoría + fecha).
- Pienses el algoritmo de inserción manteniendo el orden (búsqueda de la posición + desplazamiento, o archivo temporal).
- Declares los parámetros de la función con el tipo correcto (por valor si es una copia, por referencia/puntero si vas a modificar el archivo o una variable externa).

6.3 Ejercicio resuelto: función de Derivación (final 22/07/2024, CONAII SI)

“Los evaluadores son 20 por categoría (los primeros 20 para la categoría 1, los siguientes 20 para la 2, etc.). Cada trabajo se asigna a 3 evaluadores, en cantidades similares.”

Idea de resolución:

1. Calcular el rango de evaluadores de la categoría: `desde = (categoria-1)*20 + 1`, `hasta = categoria*20`.
2. Mantener un **puntero rotativo** (contador) por categoría que indique “cuál es el próximo evaluador a asignar” dentro de ese rango — así las asignaciones quedan parejas (se reparten en round-robin).
3. Por cada trabajo de esa categoría sin evaluadores asignados, tomar los siguientes 3 evaluadores disponibles en el rango, asignarlos, y avanzar el puntero rotativo (volviendo a `desde` si se pasa de `hasta`).

```
void derivacion(RegistroTrabajo trabajos[], int cantidadTrabajos,
                int proximoEvaluadorPorCategoria[6]) {
    for (int i = 0; i < cantidadTrabajos; i++) {
        int cat = trabajos[i].categoria;
        int desde = (cat - 1) * 20 + 1;
        int hasta = cat * 20;
        for (int e = 0; e < 3; e++) {
            trabajos[i].idEvaluadores[e] = proximoEvaluadorPorCategoria[cat];
            proximoEvaluadorPorCategoria[cat]++;
            if (proximoEvaluadorPorCategoria[cat] > hasta)
                proximoEvaluadorPorCategoria[cat] = desde;
        }
    }
}
```

(`proximoEvaluadorPorCategoria` se declara y se inicializa aparte, una posición por categoría, arrancando en el `desde` de cada una — es una estructura auxiliar que hay que declarar explícitamente, como pide la consigna.)

6.4 Ejercicio resuelto: función Dictamen (final 29/07/2024, CONAISI)

“Complete el campo resultado según las notas del evaluador, usando las tablas de puntaje.”

Tablas del enunciado (para memorizar la lógica, no los números exactos — en el final te los vuelven a dar):

Dictamen	Aspectos Formales (máx 30)	C&T (máx 60)	Autores (máx 20)
Aprobado totalmente	> 22	≥ 35	≥ 17
Aprobado, requiere modificaciones	> 20	≥ 30	≥ 25 (¡ojo, dato del enunciado!)
No aprobado, reformular	> 17	≥ 25	≥ 12
Rechazado	menor al anterior, o 0 en cualquier ítem		

```
void dictamen(Lista trabajosPorCategoria[], int puntajes[12],
              int idEvaluador, int categoria, char titulo[50]) {
    // 1. Sumar los puntajes por bloque (aspectos formales / C&T / autores)
    int aspectosFormales = 0, cienciaYTecnologia = 0, autores = 0;
    for (int i = 0; i < 4; i++) aspectosFormales += puntajes[i];
```

```

for (int i = 4; i < 8; i++) cienciaYTecnologia += puntajes[i];
for (int i = 8; i < 12; i++) autores += puntajes[i];

// 2. Chequear rechazo automático (algún ítem en 0)
bool hayRechazoAutomatico = false;
for (int i = 0; i < 12; i++)
    if (puntajes[i] == 0) hayRechazoAutomatico = true;

// 3. Buscar el nodo del trabajo (por categoría, título) y setear su dictamen
Nodo *trabajo = buscarEnLista(trabajosPorCategoria[categoria-1], titulo);
if (trabajo != NULL) {
    if (hayRechazoAutomatico)
        strcpy(trabajo->dato.dictamen, "Rechazado");
    else if (aspectosFormales > 22 && cienciaYTecnologia >= 35 && autores >= 17)
        strcpy(trabajo->dato.dictamen, "Aprobado totalmente");
    else if (aspectosFormales > 20 && cienciaYTecnologia >= 30 && autores >= 25)
        strcpy(trabajo->dato.dictamen, "Aprobado, requiere modificaciones");
    else if (aspectosFormales > 17 && cienciaYTecnologia >= 25 && autores >= 12)
        strcpy(trabajo->dato.dictamen, "No aprobado, reformular");
    else
        strcpy(trabajo->dato.dictamen, "Rechazado");
}
}

```

Por qué se resuelve así: el patrón general de estos problemas de cátedra es siempre el mismo → (1) sumar/derivar datos según una tabla de reglas, (2) aplicar las reglas en orden de mayor a menor exigencia con `if/else if`, (3) ubicar el registro a modificar dentro de la estructura (vector, lista o archivo) y actualizarlo. Reconocer este patrón te permite resolver cualquier variante (otro año, otra tabla) con la misma lógica.

7. Recursividad (por si toma un ejercicio corto)

- Toda función recursiva necesita **caso base** (corta la recursión) y **caso recursivo** (se acerca al caso base).
- Es muy usada con listas enlazadas (recorrer/buscar/insertar recursivamente) como alternativa a los `while`.

```
int contarNodos(Nodo *l) {  
    if (l == NULL) return 0;           // caso base  
    return 1 + contarNodos(l->siguiente); // caso recursivo  
}
```

8. Checklist de repaso para el 24/07

- ☐ Puedo declarar un `struct` con los campos de un enunciado nuevo sin dudar.
- ☐ Sé diferenciar cuándo pasar un parámetro por valor vs. por referencia/puntero, y explicar por qué.
- ☐ Puedo programar de memoria: `push`, `pop`, `queue`, `unQueue`, `insertarOrdenado`, `buscarEnLista`, eliminar de una lista.
- ☐ Sé resolver “recorrer estructura A y actualizar estructura B” (vector ↔ lista ↔ archivo).
- ☐ Sé aplicar una tabla de reglas de negocio con `if/else if` en el orden correcto (de más exigente a menos).
- ☐ Puedo trazar a mano un programa con punteros y arrays dinámicos, línea por línea, sin correr el código.
- ☐ Repasé los V o F típicos sobre paso de parámetros y punteros.
- ☐ Entiendo la diferencia entre archivo secuencial y de acceso directo, y cómo mantener el orden al insertar.
- ☐ Practiqué al menos un final completo de punta a punta, cronometrado.

9. Recursos que armaste / que tenés disponibles

- **Clases de cátedra 2023** (Google Drive): <https://drive.google.com/drive/folders/1AEg2Ow0umRQHaxHPXpBdAxjTFb0zCbow>
- **Clases de Yamila (2022)**, YouTube: <https://www.youtube.com/playlist?list=PLJ3LbtFnRBJgw8laexNxmJYQTRUWO2qIJ>
- **Clases de Méndez (2020)**, YouTube: https://www.youtube.com/playlist?list=PLgLCImfmlabt_pXx-DRYytGXPeY0DKkzw
- **Finales resueltos (GitHub, 2018)**: <https://github.com/aleeherasimiuk/AyED2018>
- **Página de finales de Bruno**: <https://droscarbruno.wordpress.com/finales-algoritmos-2/>

Recomendación: mirá 2-3 finales resueltos del repo de GitHub y de la página de Bruno para tomarte el tiempo real de escritura a mano — en el final no tenés compilador para probar.